OS-U4-ORANGE PROBLEM

Name: Kurumeti Mitesh

SRN: PES2UG24AM080

SECTION: AIML-4B

GitHub link: https://github.com/Mitxh13/PES2UG24AM080-pes-vcs

Phase 1

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ xxd .pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2855130c240ff131f70c626442 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
```

Phase 3

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ echo "hello" > file1.txt && echo "world" > file2.txt
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./pes add file1.txt file2.txt
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  test_objects
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  tree.h
  untracked:  README.md
  untracked:  tree.c
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  pes.h
  untracked:  commit.c
  untracked:  index.c
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776407248 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776407248 6 file2.txt
```

Phase 4

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ ./pes log
commit 6c6792734099cce1013c9cbfbf4bbb86c5c3d1aaff2cf2f92d318b78674ab518
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776407956

    Add farewellr auth!

commit e4c73cd1ae37f2f8fe20d72a8acd875f8d52c61622361f01594266560a96b22b
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776407948

    Add worldellr auth!

commit 1710fb8c283b791c92524c15c6ff352e6846d0a77f0838a40cbf1c4120f2ed33
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776407936

    Initial committial commitEA
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/17/10fb8c283b791c92524c15c6ff352e6846d0a77f0838a40cbf1c4120f2ed33
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6c/6792734099cce1013c9cbfbf4bbb86c5c3d1aaff2cf2f92d318b78674ab518
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e4/c73cd1ae37f2f8fe20d72a8acd875f8d52c61622361f01594266560a96b22b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343cB4df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ cat .pes/refs/heads/main && cat .pes/HEAD
6c6792734099cce1013c9cbfbf4bbb86c5c3d1aaff2cf2f92d318b78674ab518
ref: refs/heads/main
```

```
mitesh@UbuntuVM:~/Desktop/OS_orange/PES2UG24AM080-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 3af0223403df... Initial commit

Log after first commit:
commit 3af0223403df5b7b923c137484c238761e0d24428330c5e6a309ae72a8475099
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776408072

    Initial commit
```

```
--- Second Commit ---
Committed: 95fd6f1563ca... Update file.txt

--- Third Commit ---
Committed: 768b7b040956... Add farewell

--- Full History ---
commit 768b7b0409566d8650d527f93b26d57a885985de144bde10ec4404255ec89e05
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776408073

    Add farewellr auth!

commit 95fd6f1563ca3bfd76e74706698cccf552a0c92d7b0e7d9cef080b931f52d300
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776408073

    Update file.txtuth!

commit 3af0223403df5b7b923c137484c238761e0d24428330c5e6a309ae72a8475099
Author: Kurumeti Mitesh <PES2UG24AM080>
Date:   1776408072

    Initial committial commitEA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
768b7b0409566d8650d527f93b26d57a885985de144bde10ec4404255ec89e05

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/3a/f0223403df5b7b923c137484c238761e0d24428330c5e6a309ae72a8475099
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/76/8b7b0409566d8650d527f93b26d57a885985de144bde10ec4404255ec89e05
.pes/objects/95/fd6f1563ca3bfd76e74706698cccf552a0c92d7b0e7d9cef080b931f52d300
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5 & 6:

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answers)

A5.1 (Checkout): Update .pes/HEAD to point to the new branch. Restore every file in the working directory from the target commit's tree — read each blob and write it to disk. This is

complex because it must handle files that exist in one branch but not another (delete them), and must update the index to match.

A5.2 (Dirty working directory): For each file in the index, compare its stored mtime/size to the working directory's current stat. If they differ, recompute its blob hash and compare to the index hash. If any tracked file would be overwritten by the branch switch AND has local changes, abort.

A5.3 (Detached HEAD): In detached HEAD, new commits are made but no branch pointer advances to track them. If you switch away, those commits become unreachable. You can recover them if you remember the commit hash (it was printed when you committed), by creating a branch: git branch recovery-branch <hash>.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answers)

A6.1 (GC): Start from every branch ref and HEAD. BFS/DFS through the commit → tree → blob graph, adding every visited hash to a reachable hash set. Then walk all .pes/objects/ files

— any hash not in the set can be deleted. For 100,000 commits with 50 branches, you'd visit roughly 100,000 commits + their trees and blobs (potentially millions of objects).

A6.2 (GC race condition): A concurrent commit might: (1) write a new blob, (2) GC scans and doesn't see it referenced yet, (3) GC deletes it, (4) commit tries to reference it — now the object is gone. Git avoids this with a grace period (objects younger than 2 weeks are never deleted) and by writing objects before referencing them.